

Travaux pratiques – Chiffrer et déchiffrer des données avec OpenSSL

(Version de l'instructeur)

Remarque à l'intention de l'instructeur : le texte en rouge ou surligné en gris apparaît uniquement dans la version de l'instructeur.

Objectifs

Partie 1 : messages de chiffrement avec OpenSSL

Partie 2 : messages de déchiffrement avec OpenSSL

Contexte/scénario

OpenSSL est un projet open source qui fournit une boîte à outils robuste, de qualité commerciale et complète pour les protocoles TLS (Transport Layer Security) et SSL (Secure Sockets Layer). C'est aussi une bibliothèque de cryptographie à usage général. Au cours de ces travaux pratiques, vous allez utiliser OpenSSL pour chiffrer et déchiffrer des messages texte.

Remarque: bien que OpenSSL soit la bibliothèque de cryptographie par excellence, l'utilisation qui en est faite ici n'est PAS recommandée dans la cadre d'un système de protection renforcé. Ces travaux pratiques présentent deux problèmes de sécurité :

- 1) La méthode décrite ici utilise une fonction de dérivation de clés faible. La sécurité N'est assurée QUE par un mot de passe très fort.
- 2) La méthode décrite ici ne garantit pas l'intégrité du fichier texte.

Ces travaux pratiques doivent uniquement être utilisés à des fins de formation. Les méthodes présentées ici ne doivent pas être utilisées pour sécuriser des données sensibles réelles.

Ressources requises

- Poste de travail CyberOps VM

Instructions

Partie 1 : Chiffrer des messages avec OpenSSL

OpenSSL peut être utilisé comme outil autonome pour le chiffrement. Même si de nombreux algorithmes de chiffrement peuvent être utilisés, nous utiliserons ici principalement AES. Pour utiliser AES afin de chiffrer un fichier texte directement à partir de la ligne de commande avec OpenSSL, procédez comme suit :

Étape 1: Chiffrer un fichier texte

- a. Ouvrez une session dans le poste de travail virtuel CyberOPS.
- b. Ouvrez une fenêtre de terminal.
- c. Le fichier texte à chiffrer étant dans le répertoire `/home/analyst/lab.support.files/`, accédez à ce répertoire :

```
[analyst@secOps ~]$ cd ./lab.support.files/  
[analyst@secOps lab.support.files]$
```

- d. Saisissez la commande suivante pour afficher à l'écran le contenu du fichier texte **lettre_à_grand-mère.txt** chiffré :

```
[analyst@secOps lab.support.files]$ cat lettre_to_grandma.txt
```

Bonjour Grand-mère,

Je t'écis cette lettre pour te remercier pour les cookies au chocolat que tu m'as envoyés. Je les ai reçus ce matin et j'ai déjà mangé la moitié de la boîte ! Ils sont vraiment délicieux !

Je t'envoie de tendres pensées. Bien à toi,
Ton petit-fils qui adore les cookies.

```
[analyst@secOps lab.support.files]$
```

- e. Depuis la même fenêtre de terminal, lancez la commande suivante pour chiffrer le fichier texte. Cette commande utilise AES-256 pour chiffrer le fichier texte et enregistre la version cryptée comme **message.enc**. OpenSSL vous demande de définir un mot de passe et de confirmer ce mot de passe. Définissez un mot de passe demandé et mémorisez-le.

```
[analyst@secOps lab.support.files]$ openssl aes-256-cbc -in lettre_à_grand-  
mère.txt -out message.enc
```

enter aes-256-cbc encryption password:

Verifying - enter aes-256-cbc encryption password:

```
[analyst@secOps lab.support.files]$
```

Documenter le mot de passe.

Choix de mot de passe de l'étudiant

- f. Lorsque le processus est terminé, utilisez la commande **cat** pour afficher le contenu du fichier **message.enc**.

```
[analyst@secOps lab.support.files]$ cat message.enc
```

Le contenu du fichier **message.enc** s'est-il affiché correctement ? À quoi ressemble-t-il ? Expliquez votre réponse.

Non. Le fichier semble corrompu, car seulement quelques symboles apparaissent. Ces symboles apparaissent, car OpenSSL a généré un fichier binaire.

- g. Pour rendre le fichier lisible, exécutez de nouveau la commande OpenSSL, mais cette fois-ci ajoutez l'option **-a**. L'option **-a** indique à OpenSSL de coder le message chiffré avec une méthode de codage **Base64** avant de stocker les résultats dans un fichier.

Remarque : Base64 est un groupe de modèles de codage binaire vers texte semblables utilisés pour représenter des données binaires dans un format de chaîne ASCII.

```
[analyst@secOps lab.support.files]$ openssl aes-256-cbc -a -in  
lettre_à_grand-mère.txt -out message.enc  
enter aes-256-cbc encryption password:  
Verifying - enter aes-256-cbc encryption password:
```

- h. Utilisez de nouveau la commande **cat** pour afficher le contenu du fichier **message.enc** de nouveau généré :

Remarque : le contenu du fichier **message.enc** varie.

```
[analyst@secOps lab.support.files]$ cat message.enc  
U2FsdGVkX19ApWyrn8RD5zNp0RPCuMGZ98wDc26u/vmj1zyDXobGQhm/dDRZasG7  
rfnth5Q8NHValEw8vipKGM6dNFyYr9/hJUzCoqhFpRHgNn+Xs5+T0tz/QCPN1bi  
08LGTSzOpfkg76XDck8uPy1hl/+Ng92sM5rgMzLXfEXtaYe5UgwOD42U/U6q73pj  
a1ksQrTWsv5mtN7y6mh02Wobo3AlooHrM7niOwK1a3YKrSp+ZhYzVTrtksWDl6Ci  
XMufkv+FOGn+SoEEuh7l4fk0LIPEfGsExVFB4TGdTizQApRw74rTAZaE/dopaJn0
```

```
sJmR3+3C+dmgzZIKEHwsJ2pgLvj2Sme79J/XxwQVNpw=  
[analyst@secOps lab.support.files]$
```

Le fichier **message.enc** s'est-il affiché correctement ? Expliquez votre réponse.

Oui Bien que message.enc soit chiffré, il s'affiche maintenant correctement, car il a été converti du format binaire au format texte et encodé en base64.

Quel est selon vous l'intérêt d'avoir codé **message.enc** en Base64 ?

Le message chiffré peut maintenant être copié et collé dans un message électronique, par exemple.

Partie 2 : Déchiffrer des messages avec OpenSSL

Avec une commande OpenSSL similaire, il est possible de déchiffrer **message.enc**.

- a. Utilisez la commande ci-dessous pour déchiffrer **message.enc** :

```
[analyst@secOps lab.support.files]$ openssl aes-256-cbc -a -d -in message.enc  
-out lettre_déchiffrée.txt
```

- b. OpenSSL vous demande le mot de passe utilisé pour chiffrer le fichier. Saisissez de nouveau le même mot de passe.
- c. Quand OpenSSL finit de déchiffrer le fichier **message.enc**, il enregistre le message déchiffré dans un fichier texte appelé **lettre_déchiffrée.txt**. Utilisez la commande **cat** pour afficher le contenu du fichier **lettre_déchiffrée.txt** :

```
[analyst@secOps lab.support.files]$ cat decrypted_letter.txt
```

La lettre a-t-elle été correctement déchiffrée ?ez vos réponses ici

Oui, la lettre a été déchiffrée correctement.

La commande utilisée pour déchiffrer contient également une option -a. Pouvez-vous expliquer pourquoi ?

Comme message.enc a été encodé en base64 après le processus de chiffrement, message.enc doit être décodé en base64 pour pouvoir être déchiffré par OpenSSL.